

 Member-only story

How to Set Up and Use a Vector DB In Less Than 10 Minutes

Speed up your vector searches when high latency isn't an option

7 min read · Feb 14, 2025



Emily Gibbs

Follow

 Listen

 Share

... More

Have you ever wondered how large companies like DoorDash, Pinterest, Spotify, and Airbnb provide search and recommendation capabilities with millions of listings for millions of customers, while maintaining negligible latency? As a data science and engineering consultant with over 5 years of experience solving these kinds of data problems, I'm going to introduce you to the technology behind that magic — *vector databases* — and help you set up your own proof-of-concept (POC) vector search as a recommendation method in less than 10 minutes! By the end of this article you will:

1. Understand what a vector database is.
2. Know how vector databases are distinct from relational databases.
3. Set up your own vector database locally on your machine.
4. See the difference that using a vector database can make in vector search speed.

(Psst this article assumes you already understand the basics of *vector embeddings*. If you don't yet have a good grasp of what a vector is or what it means for vectors to be similar, I recommend checking out [this approachable explanation](#) before continuing)

For the duration of this article, imagine you run a research community website where people read and share research papers. Your tabular data may look something like this:

title	doi	author	publication_date	abstract
A Longitudinal Study of Socks Lost in Laundry: The Case for a Parallel Universe	1000.10000	John Smith	01/01/2022	Over a 10-year study involving 1,024 laundry cycles, we observed a statistically significant disappearance of socks, with no correlation to machine type, load size, or detergent brand. Quantum entanglement theories suggest the existence of a parallel universe populated exclusively by single socks. Further research is needed to establish inter-dimensional laundry protocols.
The Aerodynamics of Toast: Why it Always Lands Butter-Side Down	2345.12345	Jane Doe	05/21/1998	Through 500 controlled toast-dropping experiments, we confirmed a 96% probability of butter-side-down landings. Analysis reveals a complex interplay of toast rotation, butter weight distribution, and Murphy's Law. We propose a novel "anti-butter" coating to mitigate breakfast-related tragedies.
The Acoustic Properties of Crunchy Snacks: Why They're Louder During Quiet Moments	1111.22222	Joe Schmo	11/05/2008	This study explores the phenomenon of snack acoustics, revealing that crunchy sounds amplify by 300% in silent environments. Psychoacoustic analysis suggests a direct link between perceived loudness and social embarrassment. Recommendations include pre-snack noise masking strategies, such as fake coughing or strategic TV volume increases.

As the business owner, here are two possible features you may wish to add to your website:

1. Give users the ability to filter the papers based upon date.
2. Show users recommended papers based on papers they have read in the past.

The first scenario is a use case for the tabular data — in the background it is a SQL query to the table we showed above with the filter `WHERE publication_date BETWEEN filter_date_start AND filter_date_end`.

But what about the second scenario? You may have a transaction table that contains the papers each user has read, but how do you find the papers that are the most similar out of the millions of possible options? And how do you do it quickly, for every user on your busy site?! This task is exactly the type where vectors shine, because we can determine how similar two papers are based on the calculated similarity (e.g. cosine similarity) between the vector embeddings of the paper abstracts.

HOWEVER, there are two big logistical problems with this approach that we need to solve:

1. How do we effectively store the vectors for each of the millions of papers (when vectors are frequently hundreds or thousands of features long)
2. How do we quickly search through them? Do we try to compare the papers a user read to every. other. paper (in real time) to find the most similar ones?

These logistical problems are why vector databases exist, even when we already have relational DBs, document DBs, graph DBs, etc. Vectors are a unique type of data that support a different set of use cases than the data we typically work with. A vector DB provides an effective way to store high-dimensional vectors through compression methods, and allows for much faster similarity searches through the utilization of vector indexing algorithms.

To see if a vector database will really help with our example use case, let's set one up! We're going to create a proof-of-concept (POC) for the 2nd business feature we mentioned above: recommending papers that are similar to ones that a user has already read.

Setup

For this demo, we're going to be using [Chroma](#) — an open-source vector database option that will allow us to set up a local vector database without needing to mess with additional complexities such as Docker. These are the packages that we will need to install:

Then import the following:

Get Our Data

Now that we've got our environment set up, we need to get the data we'll be using. Our dataset comes from The Alexandria Index which provides a free dataset of research papers from the popular research paper archive arXiv. For each paper this data includes:

- Abstract (chunk of text; summary of the paper)
- DOI (unique identifier that links back to the original paper)
- Vector embedding of the paper abstract (represents the semantic meaning of the abstract)

You can access this dataset [here](#). We'll be working with a subset of the data that contains 500,000 paper abstracts.

Once we have those files downloaded, let's load them into a pandas dataframe `data_df` to start working with it:

If we display the first 5 rows with `data_df.head()` we'll see that the data looks something like this:

For the sake of this example, we're going to say that our user has read and liked the research paper with the DOI `1606.02603` . Let's take a look at the abstract for that paper:

It looks like this user enjoys reading papers about robots and the interaction between robots and humans. We would want our solution to recommend papers about this topic.

Vector Search Using Brute Force

We want to make sure that adding the overhead of a new tool like a vector database is worth it compared to more simple methods. To determine that, let's set up a brute force method and see how that performs. This method will calculate the cosine similarity between the vector of the paper the user read and every other vector in the dataset to determine which have the highest similarity:

After running for awhile we get these results:

Based on the beginning snippets of the abstracts, it looks like we found very similar papers (you can print out the abstracts themselves if you'd like to check). But taking almost a whole minute to find recommendations is not going to be fast enough in a production environment. A user would likely not wait this long — and this isn't even the full dataset! Let's see if using a vector database improves these results.

Set Up the Vector DB

To set up our Chroma vector DB, we'll first instantiate a database *client*. A client lets us connect to the database. To do this locally, we just need to pass in the path to a folder where Chroma can store our database files:

Now we create a *collection* (think of this as a vector database equivalent of a table):

When we create a collection Chroma automatically creates a *vector index* in the background that will speed up the search process.

Now we add our data to the collection (in batches since there's a limit to how much data Chroma can add at a time). Don't worry if this takes awhile — the additional time is a result of setting up the index.

Vector Search Using a Vector DB

Now that our vector DB is populated, time to test its performance! Let's create a function to search the database, again trying to find papers similar to the one our example user has read:

We don't have to wait long for the results here:

The search with the vector database took only 1.084 seconds — almost 50x faster than the brute force method! Comparing the results, it appears we got the exact same top 5 results from both methods, showing that this increase in speed didn't cause a decrease in quality. Looks like the vector database is definitely worth implementing for this use case.

Congratulations! If you made it this far that means you learned about a use case vector databases are great for and how to quickly set up a local vector database for an POC recommendation method. Follow me here on Medium or on [LinkedIn](#) to get notified when I break down other key concepts!

References

How DoorDash uses vectors/vector databases for their recommendation system:

<https://careersatdoordash.com/blog/personalized-store-feed-with-vector-embeddings/?t>

How Airbnb uses vectors/vector databases for their recommendation system: <https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e>

How Spotify uses vectors/vector databases for search: <https://engineering.atspotify.com/2022/03/introducing-natural-language-search-for-podcast-episodes/>

How Pinterest uses vectors/vector databases: <https://medium.com/pinterest-engineering/pintext-a-multitask-text-embedding-system-in-pinterest-b80ece364555>

Introduction to Vectors in Machine Learning: https://medium.com/@emily_gibbs/introduction-to-vectors-596d67efc874

Chroma Vector Database: <https://www.trychroma.com>

Chroma Documentation: <https://docs.trychroma.com/docs/overview/introduction>

The Alexandria Index: <https://alex.macrocosm.so/download>

arXiv research paper distribution service and open-source archive: <https://arxiv.org>

Data Science

Database

AI

Data Engineering

Recommendation System



Following

Published in Data Science Collective

863K followers · Last published 13 hours ago

Advice, insights, and ideas from the Medium data science community



Follow

Written by Emily Gibbs

317 followers · 19 following

Hello! I'm Emily, a Data Scientist and Engineer with a passion for teaching others tech in an engaging and understandable way. www.linkedin.com/in/emgibbs

Responses (4)





Erdogan T

What are your thoughts?



Tanvir Anjum

Mar 9

...

Amazing share

 20 [Reply](#)



Simple Stack

Mar 10

...

postgres db for the win, although xata is also amazing

 15 [Reply](#)



RamNivas Singh

Mar 14

...

This is amazing, thank you for putting the steps and code to get started in VectorDB

 2  1 reply [Reply](#)

See all responses

More from the list: "LLMd"

Curated by Erdogan T



Ramakrushna Mohapatra

Top 10 LLM & RAG Projects for Your AI...

★ · Aug 3



In Coding N... by Civil Lear...

Google's New LLM Runs on Just 0.5 GB RAM— ...

★ · Aug 15



In Level Up C... by Fareed ...

Building the Entire RAG Ecosystem and...

★ · Aug 6



Sau Sheong

How to fine-tune an LLM

★ · Jun 18

>

[View list](#)

More from Emily Gibbs and Data Science Collective

 In Data Science Collective by Emily Gibbs

Creating a Semantic Image Search with Vector Embeddings

How to improve traditional image search methods

 Dec 15, 2024  343  5

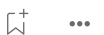


 In Data Science Collective by Rohit Patel

Understanding reinforcement learning for model training from scratch

An intuitive treatment of RLHF, TRPO, PPO, GRPO, DPO and RLAIIF

Aug 10  100  2





 In Data Science Collective by Manjunath Janardhan

SDLC is Dead. Here's What Killed It (And What's Coming Next)

I've been building software for over a decade, and I'll be honest—I'm tired of pretending the Software Development Life Cycle still...

★ Aug 2 👍 262 💬 20



 Emily Gibbs

Introduction to Vectors in Machine Learning

A primer on one of the key building blocks of multimodal AI

Dec 9, 2024 👍 341 💬 5



See all from Emily Gibbs

See all from Data Science Collective

Recommended from Medium



 In Python in Plain English by Mayuresh K

Architecture of AI-Driven Systems: What Every Technical Architect Should Know

A Step-by-Step Guide to Integrating External AI Services into Commercial Systems with Fallback Strategies

Aug 6  221  7



 In Data Science Collective by Danilo Pinto

Boosting Data Pipeline Reliability with AI and Minimal Costs

A practical guide on how to leverage AI (Ollama) in data pipelines

★ Mar 27 🖱️ 390 💬 4



 Tiya Vaj

RAG vs Vector Database

◆ Vector Database

2d ago



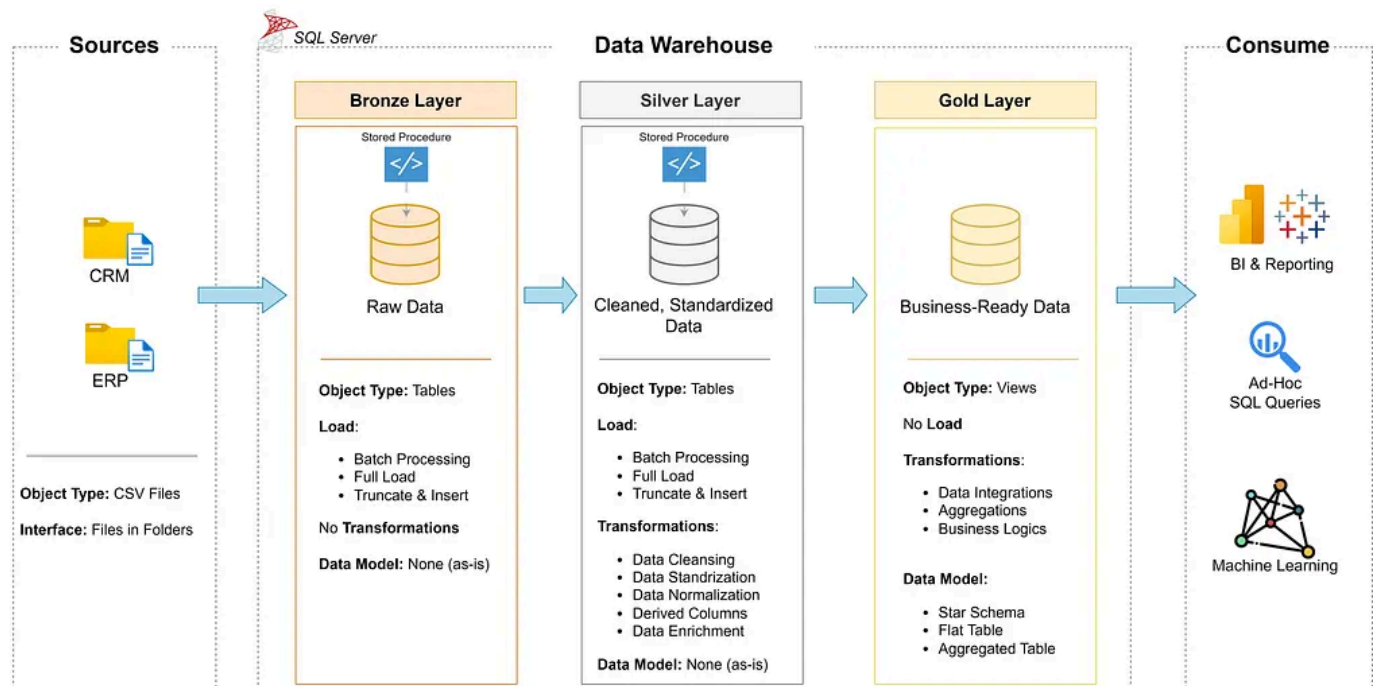


Bill Coulam

I tested 17 data modeling tools. One was the clear winner.

This will save you months by avoiding the same lengthy analysis.

★ Aug 8 🖱 119 💬 10



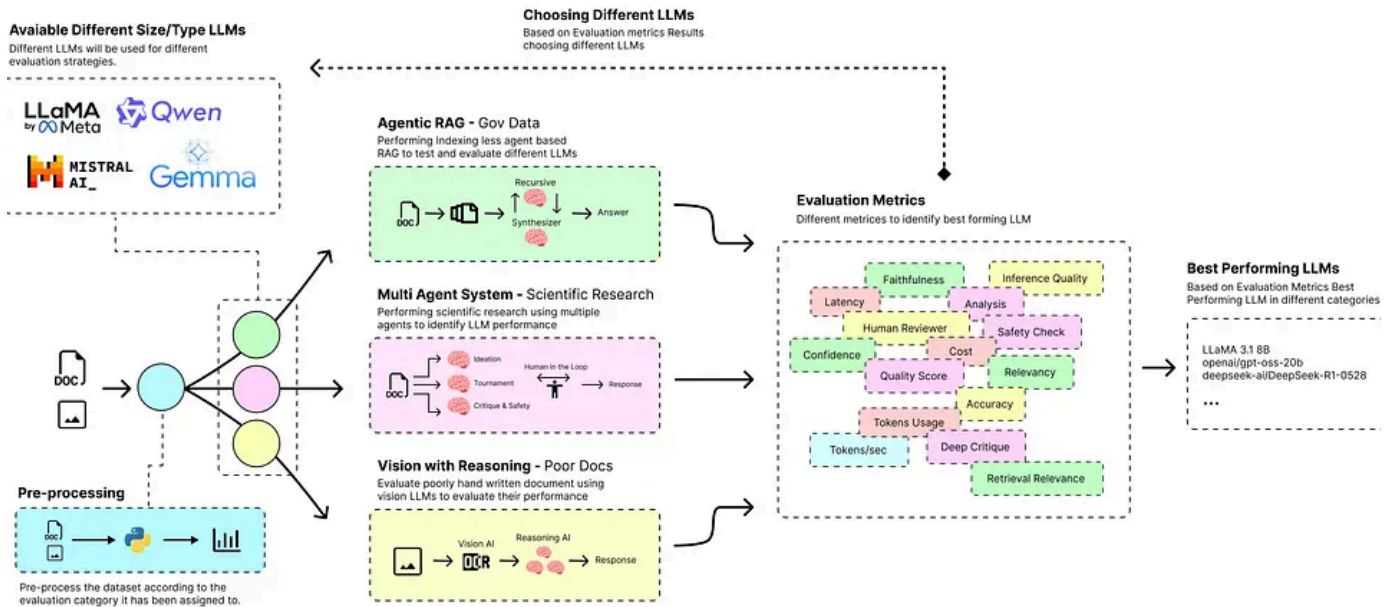
Rihab Feki

Building a modern Data Warehouse from scratch

This project demonstrates a comprehensive data warehousing and analytics solution, from building a data warehouse to generating actionable...

Apr 15 🖱 1K 💬 31





In Level Up Coding by Fareed Khan

Building Three Pipelines to Select the Right LLMs for RAG, Multi-Agent Systems, and Vision

Agentic RAG, Multi-Agent and Vision with Reasoning

★ Aug 22 🖱 826 💬 11

🔖 ...

See more recommendations